Designing executable lessons with explicit evidence boundaries

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Curriculum-design preprint · First-publication review

Source audit baseline: `52cb5457f6c485ba548cfd292f6012a941ed22cf`. Package reviewed from: `b3136006012 8d992113b602adee326e70d223340`. External source links resolve at the audit baseline; package-local files are the revised v0.1.0 candidate.

Abstract

An executable lesson can run successfully while teaching the wrong claim. A cell may compare an answer with itself, a notebook may import an obsolete implementation, or a successful finite check may be described as general proof. This guide develops a practical lesson design around five components: a declared finite example, a worked calculation, a learner prediction, a changed-input probe, and an explicit evidence boundary. It maps the 20-notebook Symbol Game integration route at a declared source baseline and works through one triangle in detail. Source inspection supports the design and its available artifacts; historical execution records are identified separately. The package adds a companion notebook that repeats the worked triangle with exact fractions only, and a pinned Lean project containing the two formal files the guide cites. No learner study, attention half-life, teaching-efficiency estimate, or transfer-effect result is reported.

1. What a lesson should let the reader do

The learner's first task is concrete: explain why an outcome changes when one declared input changes. A **declared input** is a value or rule chosen for the example. A **probe** is a particular changed input used to test a predicted behavior. A **reference check** is the computation against which the prediction is compared. An **evidence boundary** states the conclusions that this check actually supports.

For a first encounter, assume familiarity with integer arithmetic and introduce fractions before using them. Show what each symbol denotes in the current example. "Cochain," for instance, can initially mean a table assigning a rational number to each directed edge. The formal name becomes useful after the reader has worked with the table.

A complete lesson should let a reader reconstruct the original case, identify the changed input, predict an outcome with a reason, inspect the checker, and state one conclusion that the example leaves open. The notebook can record the reference computation. A record of the learner's understanding requires the learner's own response and an explicit assessment procedure.

This distinction addresses concrete defects documented by the existing curriculum review: reused generic arithmetic, constant assertions, self-comparisons, and matching LP objectives without feasibility checks. The review describes detected patterns; it does not establish that every older lesson is wrong.

2. A repeatable design sequence

Start with the smallest example that exposes the intended distinction. Write the claim before writing the notebook. "A potential change preserves a directed cycle sum" is sufficiently precise; "understand advanced topology" is not a checkable outcome for one sitting.

Then give a worked computation with each intermediate value visible. A learner should be able to follow it without first opening a distant module to learn what a variable means. Link the real implementation beside the explanation so that the source remains inspectable.

Next remove a step or vary one input. Ask for a prediction before showing the reference answer. Follow the successful case with a plausible failure: a missing bracket, a changed digest-bound atom, a dual witness that is not closed, or a forbidden numerical type. A useful probe can fail for a specific reason.

Finally, place the evidence boundary immediately after the result. A parser checks syntax. A digest check detects an inconsistency between content and its named digest. A finite certificate checker establishes its declared property for supplied data. Each deserves its own sentence.

The decision to begin with worked examples and gradually remove guidance is informed by primary studies of algebra examples and transitions from example study to problem solving: Sweller and Cooper (1985) and Renkl, Atkinson, Maier, and Staley (2002). Applying that sequence here is a curriculum-design choice. Those studies do not evaluate these notebooks, establish a universal lesson duration, or measure transfer into formal verification. The present contribution is the inspectable lesson specification and its failure controls. It is not a new learning-effect estimate or a claim that worked examples originated in this series.

3. Worked lesson: moving a discrepancy without erasing it

This lesson uses a rationally normalized version of the series' (1,1,1) triangle. It checks cycle sums and residual coordinates; no weights or quotient-minimum solver are used.

Draw vertices a, b, c with edges a to b, b to c, and c to a. Put 1/3 on every edge. Their signed cycle sum is one. A **potential** assigns a height to each vertex; the edge difference is the target height minus the source height.

Choose heights

$$f_a = 0, \qquad f_b = \tfrac{1}{3}, \qquad f_c = \tfrac{2}{3}.$$

The edge differences are

$$df = (\tfrac{1}{3}, \tfrac{1}{3}, -\tfrac{2}{3}).$$

Subtract these differences from the original edge table $h = (1/3, 1/3, 1/3)$. The residual is $h - df = (0, 0, 1)$. Its location has changed, but its cycle sum is still one.

Here is an adapted arithmetic core from the cycle-repair lesson, using the repository's discrete cochain implementation. It omits the notebook's retired `l1_norm()` call; Section 5 explains why the unchanged full notebook cannot currently complete:

```
from fractions import Fraction as Q
from omega_engine_symbol_game.domain.ddc import (
    Edge, SimplicialComplex, Cochain0, Cochain1, d0,
)

edges = (Edge("a", "b"), Edge("b", "c"), Edge("c", "a"))
carrier = SimplicialComplex(("a", "b", "c"), edges)
h = Cochain1({edge.key: Q(1, 3) for edge in edges})
f = Cochain0({"a": Q(0), "b": Q(1, 3), "c": Q(2, 3)})
adjustment = d0(carrier, f)
residual = h - adjustment

assert sum(adjustment.values.values(), Q(0)) == 0
assert sum(residual.values.values(), Q(0)) == 1
assert residual.values[("c", "a")] == 1
```

The explanation supplies a reason independent of those assertions: adding the three endpoint differences cancels every vertex height. Therefore the sum of $h - df$ equals the sum of h. The calculation instantiates that identity on a named triangle.

Prediction before the next cell

Add 7/5 to every vertex height. Predict the new edge differences and residual before reading the reference check. Identify where the added quantity appears with opposite signs.

```
shifted = Cochain0({vertex: height + Q(7, 5)
                    for vertex, height in f.values.items()})
assert d0(carrier, shifted) == adjustment
```

Both endpoints of every edge receive the same shift, so their difference is unchanged. Now negate the original edge table while keeping the same potential. Predict its new signed residual sum.

```
assert sum((-h - adjustment).values.values(), Q(0)) == -1
```

The answer changes because the edge claims changed, not because constant height shifts acquired a new effect. For a further completion exercise, leave the third original edge value blank and choose it so the cycle sum becomes zero. For a less guided task, ask the reader to construct another three-edge example and explain what a potential adjustment can preserve.

Check: the missing third edge claim is $-2/3$, since the three claims must sum to zero. For an independent example, require a stated edge orientation, exact values, and an explanation that the residual cycle sum equals the original claim cycle sum. Changing the potential cannot change that sum.

A check that can pass with the wrong sign

A cycle-sum assertion alone does not validate the edge convention. A faulty implementation that returns source height minus target height also telescopes to zero. On the displayed fixture it gives $\widetilde{df} = (-1/3, -1/3, 2/3)$, hence $h - \widetilde{df} = (2/3, 2/3, -1/3)$. This incorrect residual still has cycle sum one. The predicted coordinates $df = (1/3, 1/3, -2/3)$ distinguish these implementations. The companion notebook includes this deliberate wrong-sign control; it requires both the weak check to pass and the coordinate check to reject the mutant. A control demonstrates sensitivity to this particular fault, not to every possible implementation error.

For a learner record, retain the first predicted coordinates and explanation before revealing outputs, then record any correction separately. Stored notebook answers and an execution count cannot reconstruct that first response.

Evidence boundary

The computation checks a declared rational triangle and these particular probes. The telescoping identity explains the cycle invariant. Neither establishes a physical interpretation, neural behavior, or a learner's mastery. The code displays residual arithmetic; it does not issue a canonical native quotient-minimum receipt. Compare the practitioner guide, where replacing the claims is a separate operation.

Standalone replay

The package's companion notebook repeats this lesson with Python's `fractions` module only and imports no repository code. Its revised stored outputs come from a Python 3.12.14 execution. `tools/replay_companion_notebook.py` executes its ordinary Python cells in a fresh CPython process, then compares sequential code-cell counters and normalized output data with the deposited ones. It leaves the deposited notebook unchanged and reports its hash. The stored counts are sequential replay counters, not Jupyter kernel history. Output equality supports reproducibility of these cells in that environment; it does not validate their instructional claims. An optional Jupyter backend requires an explicitly selected kernel. Jupyter kernel replay could not complete on this review host because sockets are unavailable; no widget or IPython rendering check is claimed. The package test `code/tests/test_007_worked_triangle.py` checks the same arithmetic with plain fractions and reads the notebook's stored outputs without executing it. The same test file also runs the three code blocks of this section exactly as printed; that check needs the repository implementation and its initialization dependencies. In a standalone deposit it is skipped with that reason. The first-publication review separately executes the printed blocks against seven Git-verified DDC modules with parent-package initialization isolated; that is a focused source check, not a full repository integration run.

4. The route at the source audit baseline

The integration README defines the deep route at the source audit baseline. Its five directory labels organize progression; they are not measured age thresholds. Begin with the first two groups and use later groups when their prerequisites are available.

All 20 paths below were verified in the Git trees at the source audit baseline under `omega_engine_education/the_symbol_game/lessons/system_integration/`. The linked table is a reading map, not a claim of fresh execution. `docs/source_inventory_2026_10_02.json` retains their Git blob identities for inspection after extraction.

Group	Notebook	Main distinction
Beginners	01_marks_and_packets.ipynb	Parseable syntax versus assigned meaning
Beginners	02_lifts_and_non_descent.ipynb	Preserved old object versus a new object without an old representative
Beginners	03_edges_and_parallel_cells.ipynb	Matching boundaries versus valid syntax alone
Beginners	04_costs_and_ties.ipynb	Declared cost choice versus an unavoidable winner
Novices	01_codecs_and_tamper_evidence.ipynb	Representation preservation versus changed content
Novices	02_token_frontier.ipynb	Retained token mass versus answer correctness
Novices	03_typed_agent_composition.ipynb	Compatible types versus permission to act
Novices	04_cycle_repair.ipynb	Residual concentration versus changed cycle sum
Intermediates	01_nilpotence_and_missing_boundaries.ipynb	Complete cell boundaries versus the chain-law calculation
Intermediates	02_exact_lp_certificates.ipynb	Feasible matching witnesses versus equal objective labels
Intermediates	03_quantifier_order.ipynb	Choosing before versus after a challenge
Intermediates	04_frustrated_spin_triangle.ipynb	Finite parity obstruction versus a general physics claim
Advanced	01_four_simplex_volume.ipynb	Computed determinant versus a supplied expected constant
Advanced	02_pairing_and_degree_four_certificate.ipynb	Declared pairing versus constructed manifold topology
Advanced	03_survivor_birth_and_death.ipynb	Survivor identities versus counts alone
Advanced	04_transfer_trace_and_shadow.ipynb	Finite return counts versus broader spectral meaning
Researchers	01_coverage_and_licensed_depth.ipynb	Catalog coverage versus accepted lift evidence
Researchers	02_immune_boundary_and_negative_selection.ipynb	Bounded detection versus repair authority
Researchers	03_governed_prompt_evolution.ipynb	Selection on supplied finite outcomes versus learned model performance
Researchers	04_source_bound_research_audit.ipynb	Source navigation versus executed evidence

The retained 125-lesson syllabus supplies a broader topic ordering. The integration README explicitly avoids treating it as a second front door or treating 20 examples as implementation of all 125 topics.

5. Replay honestly after a source move

Execution evidence belongs to exact notebook and implementation bytes. The saved headless receipt reports 20 passing notebooks dated 2026-09-25. Its recorded paths use the former `omega_engine_education/notebooks/` tree. The manifest also retains 20 paths under that absent location at this baseline. These are historical records, not a current 20/20 claim.

The verifier executes ordinary Python cells, with a fresh CPython process per notebook in its aggregate mode. It does not run an IPython kernel or test rendered widgets. It checks notebook structure, selected exact-arithmetic restrictions, source hashes, and runtime assertions. A cell-count rule cannot independently establish that every cell has the instructional role its heading claims.

The cycle notebook has an additional, concrete blocker: it calls `residual.l1_norm()`, while the

pinned cochain implementation unconditionally raises `FunctionalBoundaryUnavailable` for that method. An explicit-path diagnostic bypasses the stale aggregate manifest but is expected to stop at this retired API if earlier source checks succeed:

```
python -m pip install -r omega_engine_education/the_symbol_game/lessons/
    system_integration/requirements.txt
python -B omega_engine_education/scripts/verify_system_integration.py --child
    omega_engine_education/the_symbol_game/lessons/system_integration/100_novices/04
    _cycle_repair.ipynb --output "$env:TEMP/pfp-cycle-lesson.json"
```

Use Python 3.12 or newer as the route specifies. The child command validates declared source paths and writes its selected receipt to the requested output path. Report the observed failure; do not relabel it a pass because the mathematics is sound. Section 3's adapted arithmetic core is the available example to check separately. No fresh execution of this diagnostic or the complete route is reported by the first-publication review.

A full route refresh needs reconciled manifest/source paths, maintained replacements for retired API calls, and a new receipt. Preserve the older receipt's identity. The deposited companion uses its own maintained arithmetic lesson; refreshing the external route remains separate work.

6. Audit the lesson's strongest sentence

Ask what would make the lesson's headline false. A codec lesson should alter content while keeping the old digest; checking only digest length misses the intended failure. An LP lesson should reject an infeasible witness even when the displayed objective matches. A geometry lesson should compute the determinant independently, rather than assign the expected determinant and compare it with itself.

Formal references need the same precision. GrammarCeiling.lean formalizes a bound for an integer list with both a length bound and a per-entry magnitude bound. It does not establish an unrestricted cognitive workload. GrammarDepthCeiling.lean shows that repeated unary nesting preserves supplied declaration predicates; it does not bound nesting depth or student search.

This package vendors both files, with the one module the first imports (Expressivity.lean), into a local Lean project in `lean4/` pinned to Lean 4.34.0. Each vendored file differs from its source at the audited commit only by a leading provenance comment. The project uses no external library. The historical named axiom report covers 21 explicitly selected declarations: 19 list no axioms, and the two list bounds list `propext` and `Quot.sound`. It is not a census of every environment declaration. The historical full report lists 64 package-local declarations, including generated and private names; its recorded dependencies are subsets of `propext` and `Quot.sound`. `Classical.choice` is permitted by the report's whitelist but is not listed as used. Imported Lean library declarations are outside that package-local census.

The retained 2026-10-02 receipt binds the full report and formal source hashes to a successful build and census run. This review verified those hashes and the vendored source bodies, and checked generation of the named census. It did not repeat the Lean build because Lean/Lake are unavailable on the review host. No fresh kernel-verification claim follows from reading a saved report. These statements concern the supplied finite formal models and provide no learner result.

Likewise, the package's legacy pedagogical test module checks chosen timing/metric thresholds and counts supplied execution-status fields. Those small examples are not measurements of attention,

evidence of a 90% learner pass rate, or execution of the 20-notebook route. The current guide makes none of those claims.

7. What an educational study would add

The pedagogy audit explicitly reports no learner corpus connecting participants with notebook assertions. Its synthesized research narrative failed citation validation, so it is used here as a repository audit pointer rather than as a validated literature synthesis.

A future study could compare lesson formats on predefined graph and certificate tasks. It would need participant recruitment and consent, baseline knowledge, recorded first predictions, consistent feedback, independent assessment, and a delayed transfer task. Timing, exclusions, missing responses, and analysis decisions would need specification before examining results. The comparison should test an instructional choice rather than reward familiarity with local vocabulary.

Such a study remains proposed. The useful present contribution is a lesson-authoring method and a navigable set of finite examples with auditable boundaries. Software replay can check the examples; assessing what people learned is a separate investigation.

Verification for this revision

The first-publication record is docs/first_publication_validation_2026_10_02.json. It binds the tested package and source bytes to the observed commands and separates standalone tests, focused DDC execution, notebook replay, and historical Lean evidence. The review records corrections and remaining limits.

`verify_deposit.py` checks file hashes and a two-way census by default; it does not run tests or Lean unless explicitly requested. `tools/rebuild_release.py --check` checks the archive against both manifests. File identity is an integrity check, not proof of the manuscript's conclusions. Historical receipts retain their original dates, commands, and scopes.

The Landau study guide provides worked examples, a standard-library worksheet, and a prediction record. The 24-problem quiz progresses from the triangle to checker design, source custody, formal assumptions, and a proposed learner study; the instructor key gives explicit scoring. Its levels and readiness scores are instructional choices, not validated learner classifications.

References

- Sweller, J., and Cooper, G. A. (1985). The Use of Worked Examples as a Substitute for Problem Solving in Learning Algebra. *Cognition and Instruction*, 2(1), 59–89. DOI 10.1207/s1532690xci0201_3.
- Renkl, A., Atkinson, R. K., Maier, U. H., and Staley, R. (2002). From Example Study to Problem Solving: Smooth Transitions Help Learning. *The Journal of Experimental Education*, 70(4), 293–315. DOI 10.1080/00220970209599510.

Companion use

- Companion 005 supplies the operator route.
- Companion 009 supplies the shared claim-review protocol.